NRLC-04: Alert & Workflow Migration

> **Series:** NRLC — New Relic to Dynatrace Migration Deep Dives | **Notebook:** 4 of 9 | **Created:** April 2026 | **Last Updated:** 10/31/2026

Overview

Alerts are the highest-stakes migration artifact — they wake people up. This deep dive covers the structural mismatch between NR alert policies and Gen3 Dynatrace, where alert routing lives in **Workflows** (not the Gen2 Alerting Profile object). It covers how NRQL conditions become anomaly detectors, why APM conditions still merit review even though Phase 17 now auto-converts them, the notification mapping (Workflow tasks instead of standalone Notification Channels), and the dual-alert window pattern that protects on-call during cutover.

Phase 17 alert coverage (post-2026-04-15): the engine now auto-converts **NRQL conditions**, **NRQL baseline / outlier conditions** (`baseline_alert_transformer`), **Infrastructure / Synthetic / Browser / Mobile / External-service / Multi-location-synthetic conditions** (`non_nrql_alert_transformer`), **mute rules + scheduled + recurring maintenance windows** (`maintenance_window_transformer`), **lookup-table WHERE-IN subqueries** (`lookup_table_transformer` → Resource Store JSONL + DQL `lookup` subquery), and **deployment markers / change events** (`change_tracking_transformer` → DT events API `CUSTOM_DEPLOYMENT` / `CUSTOM_CONFIGURATION`). **Phase 18** adds `aiops_transformer` for NR AI Workflows / destinations / enrichments (with the NR↔DT "Workflow" name-collision callout). See §7 of the coverage matrix (migration utilities GitHub) for the full alert-family row-set.

Table of Contents

1. NR Alert Model
2. DT Alert Model
3. Policy → Workflow

4. NRQL Condition → Metric Event
5. APM Condition → Dynatrace Intelligence Adaptive Baseline
6. Notification Channel Mapping
7. Dual-Alert Window Pattern
8. Mute Rules & Maintenance Windows

Prerequisites

Requirement	Details
Audience	On-call leads, SRE, platform engineers wiring alert paths
Standalone	This notebook is self-contained for alert + workflow migration. No required prerequisite reading.
Optional depth	NRLC-02 (full NRQL→DQL compiler), NRLC-08 (validation), NRLC-09 (toolchain)
High-stakes warning	Alert migration affects on-call. Plan a dual-alert window of 1–2 weeks before NR alerts are silenced.

Embedded Translation Context — Alert Condition Query Patterns

Each NRQL alert condition contains a query that becomes a DQL Metric Event. The patterns below cover the vast majority of alert conditions — enough to migrate alerts without consulting the full NRLC-02 compiler reference.

Threshold on simple count

```
-- NRQL
SELECT count(*) FROM TransactionError WHERE appName = 'checkout'
-- threshold: above 10 for at least 5 minutes
```

```
// DQL (Metric Event query)
fetch spans, from:-5m
| filter dt.service.name == "checkout" and isNotNull(endpoint.name)
| filter span.status_code == "error"
| summarize count = count()
// threshold: count &gt; 10
```

NR `TransactionError` becomes request spans (`isNotNull(endpoint.name)`) whose `span.status_code` is `"error"` . Do not translate it as `isNotNull(error)` : `error` is not a Dynatrace field, so that filter matches only spans where some instrumentation set a custom `error` attribute. On the validation tenant it counted 0 failures on a service with 235 of 749 requests failing in five minutes (10/05/2026) — an alert built on it never fires. For service-level failure counts the `dt.service.request.failure_count` metric is the alternative.

Percentile threshold

```
-- NRQL
SELECT percentile(duration, 95) FROM Transaction WHERE appName = 'api'
-- threshold: above 1.0 for at least 5 minutes
```

```
fetch spans, from:-5m
| filter dt.service.name == "api" and isNotNull(endpoint.name)
| summarize p95 = percentile(duration, 95)
// threshold: p95 &gt; 1s
```

Compare durations with a duration literal (`1s`). `duration("1s")` does not parse — `duration()` takes a number, not a string.

Error rate threshold

```
-- NRQL
SELECT percentage(count(*), WHERE error) FROM Transaction
-- threshold: above 5 (= 5%) for 5 minutes
```

```
fetch spans, from:-5m
| filter isNotNull(endpoint.name)
| summarize error_pct = 100.0 * countIf(span.status_code == "error") / count()
// threshold: error_pct &gt; 5
```

Translation Confidence for Alerts

Condition pattern	Confidence	Notes
Static threshold on count/avg/sum	HIGH	Direct
Percentile threshold	HIGH	`percentile(field, N)` syntax
Percentage threshold	HIGH	`100.0 * countIf(x) / count()`
Baseline (NR adaptive)	MEDIUM → LOW	Prefer Dynatrace Intelligence adaptive baselines instead of porting
APM condition (built-in error rate, etc.)	LOW	Manual: configure baseline

1. NR Alert Model

NR's alert model is a 3-tier structure:

```
Alert Policy
  ├─ Alert Condition (NRQL, APM, Infrastructure, Synthetic, ...)
  │   ├─ Threshold (warning, critical)
  │   └─ Duration / Aggregation
  └─ Notification Channel (Email, Slack, PD, Webhook, ...)
```

Conditions are typed:
- **NRQL Condition** — query + threshold; most flexible, most common
- **APM Condition** — prebuilt detection (error rate, response time, throughput)
- **Infrastructure Condition** — host/process metrics with thresholds
- **Synthetic Condition** — monitor-result-based
- **External Service Condition** — cross-service detection

The migration target depends on the source condition type.

2. DT Alert Model

Dynatrace Gen3 splits NR's monolithic alert into two layers, with **Workflows** replacing the Gen2 Alerting Profile + Notification Channel pattern:

```
Metric Event (the detection)
  | schema: builtin:anomaly-detection.metric-events (classic) /
builtin:davis.anomaly-detectors (Gen3)
  | query (DQL or metric expression)
  | threshold + operator
  | duration (n out of m samples)
  └──── raises a Detected Problem (event)
              |
              └── Workflow (the routing + action layer)
                    | trigger: detected problem event
                    | filter: severity, entity attributes, tags, bucket
                    └── Tasks (run in sequence or parallel):
                          | Slack Send message / Send email / PagerDuty Send
event
                          | HTTP Request / Run JavaScript
                          └── trigger downstream automation
```

The decoupling is intentional: a single detected problem can match multiple Workflows (different teams react differently). Notification *delivery* lives inside Workflow tasks rather than as a separate object — this is the Gen3 model. The legacy Alerting Profile + Notification Channel pattern still exists for backward compatibility but is **not** the recommended Gen3 path.

3. Policy → Workflow

An NR Alert Policy + its Notification Channels maps to a Gen3 **Workflow** in Dynatrace.

NR Concept	Gen3 Workflow Equivalent
`name`	Workflow name
`incidentPreference` (PER_POLICY / PER_CONDITION / PER_CONDITION_AND_TARGET)	Workflow trigger filters + grouping settings
Channel binding (channel.id list)	Workflow tasks (one task per delivery target)
Channel-level filters	Filter step on the detected problem trigger

The `AlertTransformer` creates one Workflow per NR policy:

- **Trigger:** detected problem event with filter conditions matching the policy's intended scope (severity, affected entities by attribute, bucket scope)
- **Tasks:** one per original notification channel (e.g., a policy with Email + Slack + PagerDuty becomes one Workflow with three tasks)
- **Conditional branches:** for `PER_CONDITION` policies, separate trigger filters per condition; for `PER_POLICY` grouping, a single trigger covers all conditions

Delivery targets are deduplicated at the **task** level, not at a separate Notification-object level. Two Workflows that both send to the same Slack channel each name that channel in their own Slack *Send message* task (`dynatrace.slack:slack-send-message`); the bot token behind it lives once, in the Slack connection both tasks reference (§6).

4. NRQL Condition → Metric Event

Each NR NRQL condition becomes a DT Metric Event. The translation pipeline:

```
NR NRQL Condition
      ↓
  [Compile NRQL → DQL via nrql-engine]
      ↓
  [Map threshold + operator]
      ↓
  [Map duration (e.g., 5 minutes) → evaluation window]
      ↓
  Emit DT Metric Event
```

Threshold mapping:

NR	DT
`thresholdType: STATIC` + `operator: ABOVE`	`analyzer.input` : `threshold` + `alertCondition: ABOVE`
`thresholdType: BASELINE`	DT auto-adaptive baseline (recommended; configures Dynatrace Intelligence)
`terms.priority: CRITICAL`	The detector's threshold, with event template `event.type: CUSTOM_ALERT` — the engine emits `CUSTOM_ALERT` for every migrated detector (`alert_transformer.py` , `main` branch, read 10/14/2026)

NR	DT
`terms.priority: WARNING`	Used for the threshold **only when the condition has no CRITICAL term**; otherwise the warning term is not migrated. If the NR warning paged someone, add a second detector for it by hand with `CUSTOM_ALERT` and route it in the workflow. Do not give it an info event type to mark it as "lower severity": *"An info event never opens a problem on its own"*, so no problem-triggered workflow ever sees it

Duration mapping:

NR	DT
`5 minutes`	`slidingWindow: 5, violatingSamples: 5` (1-min granularity)
`at least once in 5 minutes`	`slidingWindow: 5, violatingSamples: 1`
`for at least 5 minutes`	`slidingWindow: 5, violatingSamples: 5`

> **Sources:** builtin:davis.anomaly-detectors schema (DT docs).

Confidence consideration: the NRQL must compile to HIGH-confidence DQL. MEDIUM/LOW translations should be hand-validated before becoming production alerts.

The correlation requirement NR has no equivalent for

In New Relic an alert is effectively a standalone notification: the condition fires, the policy routes it, and each firing stands on its own. Dynatrace does not work that way, and the difference is not cosmetic — it decides whether your migrated alerts consolidate or multiply.

Most Davis events carry `dt.smartscape_source.id`, the Smartscape entity ID of whatever the signal is about — 84.9% on a validation tenant over 7 days, 08/11/2026. Davis's universal correlation rule groups events that share the same `dt.smartscape_source.id` into a single problem. What happens to a detector event that leaves the field unset is **not documented** — the one documented fallback belongs to the Events API v2, where an event posted without an `entitySelector` is associated with the environment entity. On a validation tenant, events without a `dt.smartscape_source.id` mostly behaved the same way: re-checked 09/28/2026 over 1 day, **3,940 carried `affected_entity_ids` naming**

the environment entity and 691 named no entity at all. **Where they land on the environment entity, every such alert names the same entity and merges into the same problem**, and no threshold or duration tuning changes that, because the fault is attribution rather than sensitivity.

This lands at exactly one point in the pipeline above: the `by:{}` clause the compiled DQL inherits from the NRQL `FACET`.

Compiled detector groups by	Attribution	Consequence
An entity dimension the metric already carries (`dt.smartscape.service` , `dt.smartscape.host` , `dt.smartscape.k8s_deployment`)	**Available** — the event template interpolates the grouping dimension into the field, and entity-scoped built-in detectors populate it without extra work	Alerts consolidate per entity
A bare string dimension (`appName` , a hostname label, a parsed log field)	**Not available** — there is no entity in the query to attribute to	No entity of its own — on the validation tenant, most such events landed on the environment entity and merged with every other unattributed alert
Nothing — a single aggregate series with no `by:`	**Not available**	Same — one shared problem, no usable root cause

`FACET appName` is the common trap. It *looks* like an entity grouping because in NR it was the unit of alerting, but it compiles to a string dimension, and a string is not an entity. The fix is to group the compiled query on the entity dimension the metric already carries, then set the detector's event template to interpolate that dimension — AIOPS-02 §4 covers the template mechanics, AIOPS-03 §1 the correlation rules.

Audit migrated detectors before the dual-alert window (§7) closes. An unattributed detector passes NR-parity testing cleanly — the alerts do fire, on time, at the right threshold — and the damage only shows up afterwards, by which point the parallel run is over and the NR side is gone. Watch for the right symptom: **not a problem-count spike, but problems whose blast radius makes no sense** — a single problem naming unrelated services with no usable root cause, because the mis-attributed alerts landed on

the same environment entity. Measured on a validation tenant over 7 days on 07/31/2026, **36,763 of 251,792 Davis events (14.6%) carried no correlation key**. Re-measured on the same tenant over 7 days on 08/11/2026, the events that had landed on the environment entity ran at **583 firings per correlation across a single entity** — 28,004 firings collapsed into 48 correlations — against **1.1 firings per correlation** for the 192,199 events that named a real entity, and one migrated detector fired 8,005 times into a **single** correlation. AIOPS-02 §8 carries the query that finds them.

> **Sources:** Avoid overalerting (DT docs) — "The same Smartscape entity rule groups all events that share the same `dt.smartscape_source.id` field value.", Ingest an event — POST /api/v2/events/ingest (DT docs) — "If not set, the event is associated with the environment (`dt.entity.environment`) entity." (Events API v2 only), Event categories (DT docs) — *"Informational Davis events aren't sent out as alerts and no problems are opened"*. **Derived:** the three-row attribution table maps the documented same-entity grouping rule onto the `FACET → by: {}` compilation step this section already describes; its environment-entity outcome for unattributed detectors is the dated tenant observation above, not a documented contract.

5. APM / Non-NRQL Condition → Dynatrace Intelligence Adaptive Baseline

NR's APM conditions ("alert when error rate above 5% for 10 minutes") use a built-in detection model. DT solves the same problem with **Dynatrace Intelligence adaptive baselines** — but the configuration is fundamentally different:

- NR sets a **fixed threshold** with a sensitivity dial.
- DT learns the **expected baseline** for each entity and alerts on statistically significant deviation.

Phase 17 auto-conversion: `non_nrql_alert_transformer` now handles Infrastructure / Synthetic / Browser / Mobile / External-service / Multi-location-synthetic conditions, and `baseline_alert_transformer` handles NR baseline/outlier NRQL conditions. All emit `builtin:davis.anomaly-detectors` configuration — no manual porting required. Review is still recommended for threshold tuning.

NR Condition	Engine Output	Recommended DT Refinement
Error rate above N% (APM)	Dynatrace Intelligence error-rate anomaly detector	Raise/lower sensitivity based on baseline width
Response time above N ms	Dynatrace Intelligence response-time adaptive detector	Consider pairing with SLO for burn-rate alerts
Throughput drop	Dynatrace Intelligence throughput adaptive baseline	—
Apdex below N	Custom DT Apdex calculation + DQL anomaly detector (DT has no native Apdex)	Phase 19 apdex-bucket uplift raises compiler confidence to HIGH for bucketed queries
NR baseline condition	Dynatrace Intelligence adaptive baseline (direct)	—
NR outlier condition	Dynatrace Intelligence outlier detector	—
Multi-location synthetic	Dynatrace Intelligence detector with location-count expression	Adjust minimum-failed-locations threshold

Review pattern: the engine imports the signal shape (which metric, which entity scope, which direction), but Dynatrace Intelligence's adaptive nature means DT will fire on different events than NR did. The dual-alert window (§7) catches this — expect DT volume to deviate from NR volume in both directions, and tune from there.

6. Notification → Workflow Task Mapping

In Gen3, notifications are not standalone objects — they are **tasks inside a Workflow**. Each NR notification channel becomes a task in the migrated Workflow.

NR Channel	Workflow Task	Migration Notes
Email	Email: *Send email* (`dynatrace.email:send-email`)	Email addresses copy directly into task config

NR Channel	Workflow Task	Migration Notes
Slack	Slack Connector: *Send message* (`dynatrace.slack:slack-send-message`)	The task references a Slack **connection** holding a bot OAuth token — not a webhook URL. Create the connection; NR's Slack webhook is not reused
PagerDuty	PagerDuty Connector: *Send event*	The routing (integration) key lives in a PagerDuty **Events connection**, not in the task
OpsGenie	*HTTP Request* (`dynatrace.automations:http-function`), or the Jira Service Management Connector	No dedicated Opsgenie connector; the API key goes into the request — **reissue required**
ServiceNow	ServiceNow Connector (e.g. `dynatrace.servicenow:snow-create-incident`)	Instance credentials live in a ServiceNow connection
Webhook	*HTTP Request* (`dynatrace.automations:http-function`)	URL + headers + payload template migrate; **secrets do not**
User (channel)	Email: *Send email*	Resolved to user's email
xMatters	*HTTP Request* (`dynatrace.automations:http-function`)	No dedicated connector; xMatters webhook URL

Critical: every task that uses a secret (token, webhook URL, API key) **must have its secret rotated and re-entered** — in the app's connection for Slack, PagerDuty, ServiceNow and Microsoft Teams, or in the task config / Dynatrace credentials vault for *HTTP Request* tasks. The transformer can't migrate secrets — it emits placeholder values that must be filled in before the Workflow is enabled.

Workflow task ordering: *"The tasks run as a directed acyclic graph (DAG)."* Order comes from the transitions between tasks, not the order they are declared or placed: *"Rearranging the tasks alone doesn't change the logical flow."* A task runs when its predecessors finish in the state its task condition requires (the *"default condition*

configuration (success and skipped)"). Make the critical task the predecessor, and set the condition on any task that must also run after a failure.

> **Sources:** Upgrade from Classic problem notification to simple workflows (DT docs) — classic integration → workflow connector mapping, incl. Opsgenie and xMatters, Slack Connector setup (DT docs) — *"Your Dynatrace Slack Connector requires an OAuth token to authorize sending messages to Slack."*, PagerDuty Connector setup (DT docs), Dynatrace workflow samples — AGENTS.md (Dynatrace GitHub) — action identifiers `dynatrace.email:send-email`, `dynatrace.slack:slack-send-message`, `dynatrace.automations:http-function`, Build workflows (DT docs).

7. Dual-Alert Window Pattern

Never silence NR alerts on the same day DT alerts go live. Run both for 1–2 weeks.

The Pattern

1. Migrate alerts to DT, but route notifications to a **silent test channel** (e.g., a Slack channel only the SRE team sees).
2. Run for 7–14 days.
3. Compare alert volume: DT should match NR within ±10%. If higher, DT is over-alerting (tune thresholds). If lower, DT may be missing detections (review condition logic).
4. Compare alert content: same problem detected at roughly the same time on both platforms?
5. Once dual-alert volume aligns, swap DT to the production channel and silence NR for that policy.
6. Migrate next policy. Don't batch — cutover one policy at a time.

Volume Comparison Query (DQL)

```
fetch dt.davis.problems, from:-7d
| summarize problems = countDistinct(display_id), by:{event.name,
event.category}
| sort problems desc
```

Count distinct problems, not records. `fetch events | filter event.kind == "DAVIS_PROBLEM"` returns one record per problem *update*: over 24 hours on the validation tenant it returned 15,735 records for 465 distinct problems (10/05/2026), so `count()` there inflates every condition by a factor that depends on how often its problems update. Compare against NR's `Issues` count for the same period. Investigate any condition with > 25% delta.

8. Mute Rules & Maintenance Windows

Phase 17 auto-conversion via `maintenance_window_transformer`:

NR Concept	DT Equivalent	Engine Output
Mute Rule (NRQL-based)	Maintenance window scoped by a DQL filter (`builtin:maintenance-windows`)	Envelope + workflow filter — see the schema note below
Scheduled Maintenance Window	`builtin:maintenance-windows`, one-time (`once`) trigger	Envelope — see the schema note below
Recurring Window	`builtin:maintenance-windows`, recurring (`time`) trigger with a recurrence rule	Envelope — see the schema note below

> **Schema note — fix the envelope before import.** The transformer writes its envelopes with `schemaId: builtin:deployment.maintenance` (`maintenance_window_transformer.py`, `main` branch, read 10/14/2026). No such schema exists: the Settings schema list names two maintenance-window schemas and neither is that one. Retarget each envelope by hand:
>
> - **`builtin:maintenance-windows`** — the Latest Dynatrace maintenance windows. Scope is a DQL filter: *"Window scope uses DQL matcher filter criteria on Grail primary fields and tags."* Problems are still detected — *"Dynatrace always detects problems."* — so the mute happens in the workflow: add `maintenance.is_under_maintenance == false` to the **Problem trigger** filter. The engine's `suppression: DETECT_PROBLEMS_DONT_ALERT` field belongs to the classic schema and has no counterpart here. Windows cannot be back-dated: *"Retroactive maintenance windows are not supported."*

> - `builtin:alerting.maintenance-window` — classic maintenance windows, scoped by entity type, entity, entity tags or management zones, with once / daily / weekly / monthly recurrence. Use it only on a tenant still running classic alerting.
>
> **Rolling out (SaaS 1.348, rollout start 10/12/2026, staged tenant rollout):** *"During an active maintenance window, scheduled executions of selected Synthetic monitors are automatically skipped."* That covers NR mute rules written against synthetic conditions — verify the feature has reached your tenant before relying on it; until then, synthetic monitors keep running during a window.
>
> **Sources:** Settings API schemas (DT docs), builtin:maintenance-windows schema (DT docs), builtin:alerting.maintenance-window schema (DT docs), Upgrade from Classic maintenance windows (DT docs), SaaS 1.348 release notes (DT docs).

For a NRQL-based mute rule, including one that filters on metric values (e.g., "mute when load < 10"), the transformer emits a maintenance-window envelope, a workflow filter holding only a comment (`/* migrated mute rule — verify: <nrql> */`) and a warning. The tool's coverage matrix calls this a *"LOW-confidence DQL-comment translation"*. Rewrite the condition by hand in the Problem trigger.

Summary

Alert migration is the highest-risk phase: errors here wake people up. Post-Phase-17, the pattern is:

1. Auto-convert NRQL + APM + non-NRQL + baseline + outlier conditions (engine handles all via Phase 17 transformers)
2. Review imported detectors — Dynatrace Intelligence adaptive behavior differs from NR fixed thresholds
3. Recreate notification channel secrets manually (never auto-migrated)
4. Auto-convert mute rules + maintenance windows (Phase 17), then retarget each envelope to a real maintenance-window schema and add the workflow maintenance filter (§8)
5. Run dual-alert for 1–2 weeks before silencing NR — use `--canary <pct>` for production tenants
6. Cut over one policy at a time

Continue to **NRLC-05 Synthetic Monitor Migration** for monitor-type-by-monitor-type migration.

Tooling for Alert-Only Migration

End-to-end commands for migrating only alerts (Metric Events + Workflows). Notification channels become Workflow tasks; secrets must be re-entered.

```
# 1. Inventory NR alert policies + conditions + notification channels
python3 migrate.py migrate --export-only --components
alerts,notification_channels --output ./alerts-run

# 2. Translate condition queries and write a conversion report — nothing is
imported
python3 migrate.py migrate --dry-run --diff --report --components
alerts,notification_channels --output ./alerts-run

# 3. Import, routed to a silent test channel first (staging Workflow target);
#    writes ./alerts-run/rollback-manifest.json
python3 migrate.py migrate --components alerts,notification_channels --output
./alerts-run

# 4. Config-drift audit: live tenant vs the transformed baseline
#    (RENAMED / DELETED / MODIFIED / EXTRA; exits 1 on drift). Read-only; does
not query New Relic.
python3 migrate.py audit --baseline ./alerts-
run/transformed/dynatrace_config.json

# 5. Run the dual-alert window for 1–2 weeks and compare NR vs DT alert volume
by hand (§7)
# 6. Promote to production channels (re-enter secrets in Workflow tasks)
# 7. Silence NR alerts for the migrated policies
```

> **Flags checked against `migrate.py` on the tool's `main` branch (10/05/2026).** There is no `--transform-only`: `--dry-run` runs export and transform, writes `<output>/transformed/dynatrace_config.json`, and skips the import. `--diff` takes effect only on a full or dry run — with `--import-only` it is ignored, and without `--dry-run` it is computed *after* the import. `--import-only` needs `--input <dir>` and does not write a rollback manifest; a full run writes `<output>/rollback-manifest.json`. Run `python3 migrate.py migrate --help` against your checkout before relying on a flag.

Two more limits, from the same code. `--diff` matches only dashboards and management zones by name; alerts, SLOs and monitors are always reported as `CREATE`, so it cannot tell you an alert was already migrated. And the import warns that Davis anomaly detectors will fail to import unless `DYNATRACE_DETECTOR_ACTOR` (a service-user UUID) is set.

Workflow validation: the tool has no command that fires or checks a workflow. Confirm each migrated workflow from its executions during the dual-alert window (§7), and compare NR and DT alert volume there — that comparison is manual.

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source NewRelic-to-Dynatrace-Migration-Utilities, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization), and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.